

Tổng hợp truy vấn dữ liệu — ôn bảo vệ ĐATN

Tài liệu này tổng hợp các kiểu truy vấn dữ liệu tiêu biểu trong PM QLKT, sắp xếp từ dễ đến phức tạp. Mỗi mục gồm: (i) mô tả nghiệp vụ, (ii) cú pháp Prisma trong code, (iii) SQL tương đương, (iv) lý do thiết kế. Phần cuối là gợi ý các câu hỏi giám khảo có thể hỏi và cách trả lời.

0. Kiến trúc truy vấn — phải nói được khi mở bài

Toàn bộ truy vấn dữ liệu đi qua **Prisma ORM** chứ không viết SQL thuần. Lớp **Repository** đóng gói gọi Prisma; lớp **Service** chỉ điều phối logic nghiệp vụ. Khi cần bảo đảm nguyên tử tính, Service mở `prisma.$transaction(async tx => ...)` và truyền `tx` qua các Repository — đảm bảo tất cả query trong transaction cùng commit/rollback.

Schema thực thể chính: QuanNhan (quân nhân), BangDeXuat (đề xuất), DanhHieuHangNam (danh hiệu hằng năm), HoSoHangNam / HoSoNienHan / HoSoCongHien (hồ sơ suy diễn), LichSuChucVu (lịch sử chức vụ), SystemLog (nhật ký), TaiKhoan (tài khoản), CoQuanDonVi / DonViTrucThuoc (cây tổ chức 2 cấp).

1. Truy vấn CRUD đơn giản

1.1 Lấy thông tin một quân nhân kèm đơn vị, chức vụ

Mục đích: Hiển thị trang chi tiết quân nhân.

Prisma (`repositories/quanNhan.repository.ts`)

```
\NormalTok{prisma}\OperatorTok{.}\AttributeTok{quanNhan}\OperatorTok{.}\FunctionTok{findUnique}\NormalTok{({\{\}\NormalTok{\NormalTok{ where}\OperatorTok{:}\}\NormalTok{\NormalTok{ id}\OperatorTok{:}\}\NormalTok{\NormalTok{ personnelId }\}\OperatorTok{,}\NormalTok{\NormalTok{ include}\OperatorTok{:}\}\NormalTok{\NormalTok{ CoQuanDonVi}\OperatorTok{:}\}\NormalTok{\NormalTok{ KeywordTok{true}\OperatorTok{,}\}\NormalTok{\NormalTok{ }\}\OperatorTok{;}}
```

SQL tương đương

```
\KeywordTok{SELECT}\NormalTok{ q.}\OperatorTok{*}\NormalTok{, cq.ten\_don\_vi }\KeywordTok{AS}\NormalTok{ cqdv\_t}\NormalTok{\KeywordTok{FROM}\NormalTok{ QuanNhan q}\NormalTok{\KeywordTok{LEFT }\NormalTok{\KeywordTok{JOIN}\NormalTok{ CoQuanDonVi cq }\NormalTok{\KeywordTok{ON}\NormalTok{ q.co\_quan\_don\_vi }\NormalTok{\KeywordTok{LEFT }\NormalTok{\KeywordTok{JOIN}\NormalTok{ DonViTrucThuoc dv }\NormalTok{\KeywordTok{ON}\NormalTok{ q.don\_vi\_truc\_thu }\NormalTok{\KeywordTok{LEFT }\NormalTok{\KeywordTok{JOIN}\NormalTok{ ChucVu }\NormalTok{\FunctionTok{cv}\NormalTok{ }\NormalTok{\KeywordTok{ON}\NormalTok{ q.chuc }\NormalTok{\KeywordTok{WHERE}\NormalTok{ q.}\NormalTok{\KeywordTok{id}\NormalTok{ }\OperatorTok{=}\NormalTok{ $\}\DecValTok{1}\}\NormalTok{\NormalTok{;}}
```

Tại sao dùng `include` thay vì gọi 3 truy vấn riêng? Prisma sinh ra một câu SELECT có nhiều LEFT JOIN — chỉ một round-trip tới PostgreSQL, tránh N+1.

1.2 Tạo mới quân nhân kèm cập nhật bộ đếm đơn vị

Mục đích: Khi thêm quân nhân thì `so_luong` của đơn vị tăng 1.

Prisma (`services/personnel.service.ts`)

```
\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\FunctionTok{$transaction}\NormalTok{({}\KeywordTok{async}\NormalTok{\KeywordTok{const}\NormalTok{ qn }\OperatorTok{=}\NormalTok{ \ControlFlowTok{await}\NormalTok{ tx}\OperatorTok{.}\AttributeTok{\ControlFlowTok{await}\NormalTok{ tx}\OperatorTok{.}\AttributeTok{coQuanDonVi}\OperatorTok{.}\FunctionTok{update}\NormalTok{\NormalTok{ where}\OperatorTok{:}\}\NormalTok{\NormalTok{ id}\OperatorTok{:}\}\NormalTok{\NormalTok{ data}\OperatorTok{.}\AttributeTok{\NormalTok{ data}\OperatorTok{:}\}\NormalTok{\NormalTok{ so\_luong}\OperatorTok{:}\}\NormalTok{\NormalTok{ increment}\OperatorTok{ }\NormalTok{\NormalTok{ }\}\OperatorTok{;}\NormalTok{\ControlFlowTok{return}\NormalTok{ qn}\OperatorTok{;}\NormalTok{\NormalTok{ }\}\OperatorTok{;}}
```

SQL tương đương

```

\ControlFlowTok{BEGIN}\NormalTok{;;}
\KeywordTok{INSERT} \KeywordTok{INTO}\NormalTok{ QuanNhan (}\KeywordTok{id}\NormalTok{, cccd, ho\_ten, ngay\_sinh}
\KeywordTok{VALUES}\NormalTok{ (}\NormalTok{ ($)\DecValTok{1}\NormalTok{, ($)\DecValTok{2}\NormalTok{, }\OperatorTok{..}\NormalTok{
\KeywordTok{UPDATE}\NormalTok{ CoQuanDonVi }\KeywordTok{SET}\NormalTok{ so\_luong }\OperatorTok{=}\NormalTok{ so}
\KeywordTok{COMMIT}\NormalTok{;;}

```

Lý do: hai thao tác phải atomic — nếu một lỗi thì rollback cả hai để bộ đếm không bao giờ lệch với số bản ghi thực.

1.3 Tìm trùng CCCD trước khi tạo quân nhân

Prisma

```

\NormalTok{prisma}\OperatorTok{.}\AttributeTok{quanNhan}\OperatorTok{.}\FunctionTok{findUnique}\NormalTok{ (}\NormalTok{ (}\NormalTok{ where}

```

SQL tương đương

```

\KeywordTok{SELECT} \OperatorTok{*} \KeywordTok{FROM}\NormalTok{ QuanNhan }\KeywordTok{WHERE}\NormalTok{ cccd}\NormalTok{

```

Note: Cột cccd có chỉ mục UNIQUE ở schema (@unique trong schema.prisma). PostgreSQL dùng B-tree index để tìm O(log n).

2. Truy vấn có lọc + phân trang

2.1 Liệt kê quân nhân với bộ lọc

Prisma

```

\NormalTok{prisma}\OperatorTok{.}\AttributeTok{quanNhan}\OperatorTok{.}\FunctionTok{findMany}\NormalTok{ (}\NormalTok{ (}\NormalTok{
\NormalTok{ where}\OperatorTok{:}\NormalTok{ \{
\NormalTok{   AND}\OperatorTok{:}\NormalTok{ \{
\NormalTok{     filter}\OperatorTok{.}\AttributeTok{co\_quan\_don\_vi\_id} \OperatorTok{?}\NormalTok{ \{ co\_qua
\NormalTok{     filter}\OperatorTok{.}\AttributeTok{gioi\_tinh} \OperatorTok{?}\NormalTok{ \{ gioi\_tinh
\NormalTok{     filter}\OperatorTok{.}\AttributeTok{search} \OperatorTok{?}\NormalTok{ \{ ho\_ten}\NormalTok{
\NormalTok{   }\OperatorTok{,}
\NormalTok{ }\OperatorTok{,}
\NormalTok{ skip}\OperatorTok{:}\NormalTok{ (page }\OperatorTok{-}\NormalTok{ }\DecValTok{1}\NormalTok{) }\OperatorTok{*}
\NormalTok{ take}\OperatorTok{:}\NormalTok{ limit}\OperatorTok{,}
\NormalTok{ orderBy}\OperatorTok{:}\NormalTok{ \{ createdAt}\OperatorTok{:} \StringTok{\textquotesingle}desc\te
\NormalTok{ include}\OperatorTok{:}\NormalTok{ \{ CoQuanDonVi}\OperatorTok{:} \KeywordTok{true}\OperatorTok{,}\NormalTok{
\NormalTok{ }\OperatorTok{;}\NormalTok{

```

SQL tương đương

```

\KeywordTok{SELECT}\NormalTok{ q.}\OperatorTok{*}\NormalTok{, cq.ten\_don\_vi, }\FunctionTok{cv}\NormalTok{.ten\_}
\KeywordTok{FROM}\NormalTok{ QuanNhan q}
\KeywordTok{LEFT} \KeywordTok{JOIN}\NormalTok{ CoQuanDonVi cq }\KeywordTok{ON}\NormalTok{ q.co\_quan\_don\_vi\_id}
\KeywordTok{LEFT} \KeywordTok{JOIN}\NormalTok{ ChucVu }\FunctionTok{cv}\NormalTok{ }\KeywordTok{ON}\NormalTok{ q.chuc\_v}
\KeywordTok{WHERE}\NormalTok{ (}\NormalTok{ ($)\DecValTok{1}\NormalTok{, ($)\CharTok{:text} \KeywordTok{IS} \KeywordTok{NULL} \Key
\KeywordTok{AND}\NormalTok{ (}\NormalTok{ ($)\DecValTok{2}\NormalTok{, ($)\CharTok{:text} \KeywordTok{IS} \KeywordTok{NULL} \Key
\KeywordTok{AND}\NormalTok{ (}\NormalTok{ ($)\DecValTok{3}\NormalTok{, ($)\CharTok{:text} \KeywordTok{IS} \KeywordTok{NULL} \Key
\KeywordTok{ORDER} \KeywordTok{BY}\NormalTok{ q.createdAt }\KeywordTok{DESC}
\NormalTok{ OFFSET }\NormalTok{ ($)\DecValTok{4} \KeywordTok{LIMIT}\NormalTok{ ($)\DecValTok{5}\NormalTok{;}\NormalTok{

```

Note: mode: 'insensitive' ánh xạ sang ILIKE của PostgreSQL — không phân biệt hoa thường.

2.2 Lọc đề xuất theo trạng thái + năm + loại

Prisma

```

\NormalTok{prisma}\OperatorTok{.}\AttributeTok{bangDeXuat}\OperatorTok{.}\FunctionTok{findMany}\NormalTok{ (}\NormalTok{ (}\NormalTok{
\NormalTok{ where}\OperatorTok{:}\NormalTok{ \{ status}\OperatorTok{:} \StringTok{\textquotesingle}PENDING\text
\NormalTok{ include}\OperatorTok{:}\NormalTok{ \{ NguoiDeXuat}\OperatorTok{:} \KeywordTok{true}\OperatorTok{,}\NormalTok{
\NormalTok{ orderBy}\OperatorTok{:}\NormalTok{ \{ createdAt}\OperatorTok{:} \StringTok{\textquotesingle}desc\te
\NormalTok{ }\OperatorTok{;}\NormalTok{

```



```
\KeywordTok{SELECT} \DataTypeTok{DATE}\NormalTok{(createdAt) }\KeywordTok{AS}\NormalTok{ ngay, }\FunctionTok{COUNT}\NormalTok{ }\KeywordTok{FROM}\NormalTok{ SystemLog}
\KeywordTok{WHERE}\NormalTok{ createdAt }\OperatorTok{>=}\NormalTok{ NOW() }\OperatorTok{-}\NormalTok{ }\KeywordTok{GROUP}\NormalTok{ BY}\NormalTok{ }\DataTypeTok{DATE}\NormalTok{(createdAt) }
\KeywordTok{ORDER}\NormalTok{ BY}\NormalTok{ ngay;}
```

Lý do: với dataset nhỏ (~vài nghìn log/ngày) group ở JS chấp nhận được. Với dataset lớn nên dùng \$queryRaw để aggregate ở DB.

4. Phân quyền theo cây đơn vị

4.1 Lấy phạm vi đơn vị của Manager

Prisma (middlewares/unitFilter.ts)

```
\NormalTok{prisma}\OperatorTok{.}\AttributeTok{quanNhan}\OperatorTok{.}\FunctionTok{findUnique}\NormalTok{({\{\}
\NormalTok{ where}\OperatorTok{:}\NormalTok{ \{\ id}\OperatorTok{:}\NormalTok{ account}\OperatorTok{.}\AttributeTok{
\NormalTok{ select}\OperatorTok{:}\NormalTok{ \{\ co\_quan\_don\_vi\_id}\OperatorTok{:}\NormalTok{ }\KeywordTok{true}\OperatorTok{
\NormalTok{\}\}\OperatorTok{;}
```

SQL tương đương

```
\KeywordTok{SELECT}\NormalTok{ co\_quan\_don\_vi\_id, don\_vi\_truc\_thuoc\_id}
\KeywordTok{FROM}\NormalTok{ QuanNhan}
\KeywordTok{WHERE}\NormalTok{ id }\OperatorTok{=}\NormalTok{ $\DecValTok{1}\NormalTok{;}
```

4.2 Liệt kê quân nhân Manager được phép xem

Manager gắn với CQĐV → thấy tất cả quân nhân cùng CQĐV (kể cả thuộc DVTT con).

Prisma

```
\NormalTok{prisma}\OperatorTok{.}\AttributeTok{quanNhan}\OperatorTok{.}\FunctionTok{findMany}\NormalTok{({\{\}
\NormalTok{ where}\OperatorTok{:}\NormalTok{ \{\}
\NormalTok{ OR}\OperatorTok{:}\NormalTok{ [\{\}
\NormalTok{ \{\ co\_quan\_don\_vi\_id}\OperatorTok{:}\NormalTok{ managerCqdvId }\OperatorTok{,}\}
\NormalTok{ \{\ don\_vi\_truc\_thuoc}\OperatorTok{:}\NormalTok{ \{\ co\_quan\_don\_vi\_id}\OperatorTok{:}\NormalTok{ }\}
\NormalTok{\}\}\OperatorTok{,}\}
\NormalTok{\}\}\OperatorTok{,}\}
\NormalTok{\}\}\OperatorTok{;}
```

SQL tương đương

```
\KeywordTok{SELECT}\NormalTok{ q.}\OperatorTok{*}
\KeywordTok{FROM}\NormalTok{ QuanNhan q}
\KeywordTok{LEFT}\NormalTok{ JOIN}\NormalTok{ DonViTrucThuoc dv }\KeywordTok{ON}\NormalTok{ q.don\_vi\_truc\_thuoc\_id = dv.don\_vi\_truc\_thuoc\_id}
\KeywordTok{WHERE}\NormalTok{ q.co\_quan\_don\_vi\_id }\OperatorTok{=}\NormalTok{ $\DecValTok{1}
\KeywordTok{OR}\NormalTok{ dv.co\_quan\_don\_vi\_id }\OperatorTok{=}\NormalTok{ $\DecValTok{1}\NormalTok{;}
```

Note: cấu trúc cây 2 cấp đơn giản, không phải đệ quy. Nếu mở rộng nhiều cấp sẽ cần CTE đệ quy WITH RECURSIVE.

5. Truy vấn phức tạp nhất — Chuỗi danh hiệu hằng năm

Đây là trọng tâm có thể bị hỏi nhiều nhất khi bảo vệ.

5.1 Tải toàn bộ lịch sử danh hiệu của một quân nhân

Prisma (services/profile/annual.ts)

```
\NormalTok{prisma}\OperatorTok{.}\AttributeTok{danhHieuHangNam}\OperatorTok{.}\FunctionTok{findMany}\NormalTok{({\{\}
\NormalTok{ where}\OperatorTok{:}\NormalTok{ \{\ quan\_nhan\_id}\OperatorTok{:}\NormalTok{ personnelId }\OperatorTok{,}\}
\NormalTok{ orderBy}\OperatorTok{:}\NormalTok{ \{\ nam}\OperatorTok{:}\NormalTok{ }\StringTok{'\textquotesingle'}desc\textquotesingle}
\NormalTok{\}\}\OperatorTok{;}
```

SQL tương đương

```
\KeywordTok{SELECT} \OperatorTok{*} \KeywordTok{FROM}\NormalTok{ DanhHieuHangNam}
\KeywordTok{WHERE}\NormalTok{ quan\_nhan\_id }\OperatorTok{=}\NormalTok{ $}\DecValTok{1}
\KeywordTok{ORDER} \KeywordTok{BY}\NormalTok{ nam }\KeywordTok{DESC}\NormalTok{;}
```

Lý do ORDER BY nam DESC: Hàm computeChainContext duyệt từ năm hiện tại lùi về quá khứ để tính độ dài chuỗi CSTĐCS liên tục, đếm cờ BKBQP/CSTĐTQ trong cửa sổ trượt 3 năm hoặc 7 năm.

5.2 Đếm số BKBQP trong cửa sổ trượt 3 năm (xét CSTĐTQ)

Logic kết hợp Prisma + JS:

```
\CommentTok{// JS — sau khi load}
\KeywordTok{const}\NormalTok{ flagsInWindow }\OperatorTok{=}\NormalTok{ rows}
  \OperatorTok{.}\FunctionTok{filter}\NormalTok{(r }\KeywordTok{=}\textgreater{} }\NormalTok{r}\OperatorTok{.}\Att
  \OperatorTok{.}\FunctionTok{filter}\NormalTok{(r }\KeywordTok{=}\textgreater{} }\NormalTok{r}\OperatorTok{.}\Att
```

SQL tương đương (nếu chuyển hẳn xuống DB)

```
\KeywordTok{SELECT} \FunctionTok{COUNT}\NormalTok{({}\OperatorTok{*}\NormalTok{)} \KeywordTok{FROM}\NormalTok{ Dan
\KeywordTok{WHERE}\NormalTok{ quan\_nhan\_id }\OperatorTok{=}\NormalTok{ $}\DecValTok{1}
  \KeywordTok{AND}\NormalTok{ nam }\OperatorTok{\textgreater{}=}\NormalTok{ $}\DecValTok{2} \OperatorTok{-} \De
  \KeywordTok{AND}\NormalTok{ nam }\OperatorTok{\textless{}}\NormalTok{ $}\DecValTok{2}
  \KeywordTok{AND}\NormalTok{ nhan\_bkbqp }\OperatorTok{=}\NormalTok{ TRUE}\NormalTok{;}
```

Lý do làm ở JS thay vì DB: lịch sử một quân nhân thường ≤ 30 dòng — load 1 lần rồi lọc nhiều lần ở JS nhanh hơn nhiều round-trip.

5.3 Upsert hồ sơ suy diễn HoSoHangNam

Sau khi tính xong, ghi kết quả vào bảng cache.

Prisma

```
\NormalTok{prisma}\OperatorTok{.}\AttributeTok{hoSoHangNam}\OperatorTok{.}\FunctionTok{upsert}\NormalTok{({{
\NormalTok{ where}\OperatorTok{:}\NormalTok{ \{ quan\_nhan\_id }\OperatorTok{:}\NormalTok{ personnelId }\}\Opera
\NormalTok{ create}\OperatorTok{:}\NormalTok{ \{ quan\_nhan\_id }\OperatorTok{,}\NormalTok{ cstdcs\_lien\_tuc}\Opera
\NormalTok{ update}\OperatorTok{:}\NormalTok{ \{ cstdcs\_lien\_tuc }\OperatorTok{,}\NormalTok{ du\_dieu\_kien\_bk
\NormalTok{}}}\OperatorTok{;}
```

SQL tương đương (PostgreSQL có cú pháp ON CONFLICT)

```
\KeywordTok{INSERT} \KeywordTok{INTO}\NormalTok{ HoSoHangNam (quan\_nhan\_id, cstdcs\_lien\_tuc, du\_dieu\_kien\_
\KeywordTok{VALUES}\NormalTok{ (}\NormalTok{ $}\DecValTok{1}\NormalTok{, $}\DecValTok{2}\NormalTok{, $}\DecValTok{3}\NormalTok{)}
\KeywordTok{ON}\NormalTok{ CONFLICT (quan\_nhan\_id)
\NormalTok{DO } \KeywordTok{UPDATE} \KeywordTok{SET}
\NormalTok{ cstdcs\_lien\_tuc }\OperatorTok{=}\NormalTok{ EXCLUDED.cstdcs\_lien\_tuc,}
\NormalTok{ du\_dieu\_kien\_bkbqp }\OperatorTok{=}\NormalTok{ EXCLUDED.du\_dieu\_kien\_bkbqp,}
\OperatorTok{.}\NormalTok{;}
```

Note: quan_nhan_id có ràng buộc UNIQUE (1-1 với QuanNhan) — vì là bảng suy diễn.

6. Transaction phức tạp — Phê duyệt đề xuất

6.1 Phê duyệt một đề xuất chứa N quân nhân

Khi Admin bấm “Phê duyệt”, một transaction duy nhất phải: 1. Đổi status đề xuất PENDING → APPROVED 2. Ghi N bản ghi vào DanhHieuHangNam (hoặc bảng tương ứng) 3. Gắn số quyết định + đường dẫn PDF 4. Ghi SystemLog với payload trước-sau 5. Tính lại HoSoHangNam cho N quân nhân bị ảnh hưởng

Prisma (services/proposal/approve/import.ts — đơn giản hoá)

```

\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\FunctionTok{$transaction}\NormalTok{()}\KeywordTok{async}
\CommentTok{// (1) Chuyển trạng thái}
\ControlFlowTok{await}\NormalTok{ tx}\OperatorTok{.}\AttributeTok{bangDeXuat}\OperatorTok{.}\FunctionTok{update}
\NormalTok{ where}\OperatorTok{:}\NormalTok{ \{ id}\OperatorTok{:}\NormalTok{ proposalId \}}\OperatorTok{,}
\NormalTok{ data}\OperatorTok{:}\NormalTok{ \{ status}\OperatorTok{:} \StringTok{\textquotesingle}APPROVED\textquotesingle}
\NormalTok{ \}}\OperatorTok{;}

\CommentTok{// (2) Ghi từng danh hiệu}
\ControlFlowTok{for}\NormalTok{ ()}\KeywordTok{const}\NormalTok{ item }\KeywordTok{of}\NormalTok{ proposal}\OperatorTok{.}
\ControlFlowTok{await}\NormalTok{ tx}\OperatorTok{.}\AttributeTok{danhHieuHangNam}\OperatorTok{.}\FunctionTok{create}
\NormalTok{ where}\OperatorTok{:}\NormalTok{ \{ quan\_nhan\_id\_nam}\OperatorTok{:}\NormalTok{ \{ quan\_nhan\_id\_nam}
\NormalTok{ create}\OperatorTok{:}\NormalTok{ \{ }\OperatorTok{...}\NormalTok{ \}}\OperatorTok{,}
\NormalTok{ update}\OperatorTok{:}\NormalTok{ \{ }\OperatorTok{...}\NormalTok{ \}}\OperatorTok{,}
\NormalTok{ \}}\OperatorTok{;}
\NormalTok{ \}}

\CommentTok{// (3) Gắn quyết định – FileQuyếtĐịnh liên kết với danh hiệu qua so\_quyet\_dinh (không có proposalId)}
\ControlFlowTok{await}\NormalTok{ tx}\OperatorTok{.}\AttributeTok{fileQuyetDinh}\OperatorTok{.}\FunctionTok{create}
\NormalTok{ \}}

\CommentTok{// (4) Ghi log}
\ControlFlowTok{await}\NormalTok{ tx}\OperatorTok{.}\AttributeTok{systemLog}\OperatorTok{.}\FunctionTok{create}
\NormalTok{ \}}

\CommentTok{// (5) Tính lại cho từng quân nhân}
\ControlFlowTok{for}\NormalTok{ ()}\KeywordTok{const}\NormalTok{ qnId }\KeywordTok{of}\NormalTok{ affectedPerson}\OperatorTok{.}
\ControlFlowTok{await} \FunctionTok{recalcAnnualProfile}\NormalTok{ (tx)\OperatorTok{,}\NormalTok{ qnId}\OperatorTok{)}
\NormalTok{ \}}
\NormalTok{ \}}\OperatorTok{;}

```

```

\ControlFlowTok{BEGIN}\NormalTok{;}

\CommentTok{{-}{-}} (1)
\KeywordTok{UPDATE}\NormalTok{ { BangDeXuat } }\KeywordTok{SET}\NormalTok{ { status } }\OperatorTok{=} \StringTok{\texttt{}}

\CommentTok{{-}{-}} (2) - N lần INSERT/UPDATE
\KeywordTok{INSERT} \KeywordTok{INTO}\NormalTok{ { DanhHieuHangNam (}\OperatorTok{..\NormalTok{.} } }\KeywordTok{VALUES}\NormalTok{ {}}

\CommentTok{{-}{-}} (3) FileQuyetDinh liên kết với danh hiệu qua so\_quyet\_dinh, không có proposal\_id FK
\KeywordTok{INSERT} \KeywordTok{INTO}\NormalTok{ { FileQuyetDinh (so\_quyet\_dinh, file\_path, nam, ngay\_ky, nguoi)}

\CommentTok{{-}{-}} (4)
\KeywordTok{INSERT} \KeywordTok{INTO}\NormalTok{ { SystemLog (action, } }\KeywordTok{resource}\NormalTok{ {, payload) }

\CommentTok{{-}{-}} (5) - N lần upsert HoSoHangNam
\KeywordTok{INSERT} \KeywordTok{INTO}\NormalTok{ { HoSoHangNam (}\OperatorTok{..\NormalTok{.} } }\KeywordTok{VALUES}\NormalTok{ {}}

\KeywordTok{COMMIT}\NormalTok{;}

```

7. Nhật ký kiểm toán

Prisma (middleware auditLog.ts)

8.3 Quân nhân chưa nhận khen thưởng nào trong 5 năm gần nhất

```
\KeywordTok{SELECT}\NormalTok{ q.}\KeywordTok{id}\NormalTok{, q.ho\_ten}
\KeywordTok{FROM}\NormalTok{ QuanNhan q}
\KeywordTok{WHERE} \KeywordTok{NOT} \KeywordTok{EXISTS}\NormalTok{ (}
  \KeywordTok{SELECT} \DecValTok{1} \KeywordTok{FROM}\NormalTok{ DanhHieuHangNam d}
  \KeywordTok{WHERE}\NormalTok{ d.quan\_nhan\_id }\OperatorTok{=}\NormalTok{ q.}\KeywordTok{id}
  \KeywordTok{AND}\NormalTok{ d.nam }\OperatorTok{\textgreater{}}\FunctionTok{EXTRACT}\NormalTok{ (}\DataType{DATE}
\NormalTok{)}\NormalTok{)};
```

8.4 Đơn vị có nhiều BKTTCP nhất

```
\KeywordTok{SELECT}\NormalTok{ cq.ten\_don\_vi, }\FunctionTok{COUNT}\NormalTok{ (}\OperatorTok{*}\NormalTok{)}\KeywordTok{AS}
\KeywordTok{FROM}\NormalTok{ QuanNhan q}
\KeywordTok{JOIN}\NormalTok{ DanhHieuHangNam d }\KeywordTok{ON}\NormalTok{ d.quan\_nhan\_id }\OperatorTok{=}\NormalTok{ q.}\KeywordTok{id}
\KeywordTok{JOIN}\NormalTok{ CoQuanDonVi cq }\KeywordTok{ON}\NormalTok{ cq.}\KeywordTok{id} \OperatorTok{=}\NormalTok{ q.}\KeywordTok{id}
\KeywordTok{GROUP} \KeywordTok{BY}\NormalTok{ cq.}\KeywordTok{id}\NormalTok{, cq.ten\_don\_vi}
\KeywordTok{ORDER} \KeywordTok{BY}\NormalTok{ so\_bkttcp }\KeywordTok{DESC}
\KeywordTok{LIMIT} \DecValTok{10}\NormalTok{)};
```

8.5 Hoạt động hệ thống theo giờ trong ngày

```
\KeywordTok{SELECT} \FunctionTok{EXTRACT}\NormalTok{ (}\KeywordTok{HOUR} \KeywordTok{FROM}\NormalTok{ createdAt) }\FunctionTok{COUNT}\NormalTok{ (}\OperatorTok{*}\NormalTok{)}\KeywordTok{AS}
\KeywordTok{FROM}\NormalTok{ SystemLog}
\KeywordTok{WHERE}\NormalTok{ createdAt }\OperatorTok{\textgreater{}}\NormalTok{ NOW() }\OperatorTok{-}\NormalTok{ }\DataType{DATE}
\KeywordTok{GROUP} \KeywordTok{BY}\NormalTok{ gio}
\KeywordTok{ORDER} \KeywordTok{BY}\NormalTok{ gio};
```

9. Câu hỏi giám khảo có thể hỏi và cách trả lời

Q1. “Tại sao em chọn Prisma thay vì viết SQL thuần?”

Trả lời: (1) Prisma sinh kiểu TypeScript tự động từ `schema.prisma` nên mọi truy vấn có gợi ý ở thời điểm soạn thảo, giảm lỗi runtime; (2) Prisma sinh ra câu SQL tối ưu (đặc biệt với `include` — ghép JOIN trong 1 query thay vì N+1); (3) Migration tự động và có thể đảo ngược; (4) Khi cần performance cực cao, em vẫn có thể dùng `$queryRaw` (raw SQL) — nhưng thực tế chưa cần dùng trong dự án này.

Q2. “Cấu trúc cây đơn vị có 2 cấp, lỡ sau cần nhiều cấp thì sao?”

Trả lời: Hiện em mô hình hoá CQĐV (cha) — DVTT (con) chỉ 2 cấp vì đặc thù tổ chức của Học viện. Nếu mở rộng sâu hơn, em có thể chuyển sang mẫu **adjacency list + recursive CTE** của PostgreSQL: thêm cột `parent_id` self-reference, query bằng `WITH RECURSIVE`. Prisma chưa hỗ trợ recursive CTE trực tiếp nhưng có thể dùng `$queryRaw`.

Q3. “Tại sao bảng `BangDeXuat` lại có 4 cột `data_*` thay vì tách bảng?”

Trả lời: Bảy loại đề xuất có schema chi tiết khác nhau (cá nhân hằng năm cần `data_danh_hieu`, niên hạn cần `data_nien_han`, công hiến cần `data_cong_hien`...). Nếu tách 7 bảng thì query “lấy tất cả đề xuất pending” phải `UNION ALL` 7 bảng — vừa rườm rà vừa chậm. Em chọn JSON để giữ schema linh hoạt; xác thực dữ liệu chi tiết được làm ở tầng Zod và `ProposalStrategy` của backend.

Q4. “Làm sao em đảm bảo `so_luong` của đơn vị luôn khớp với số quân nhân thực?”

Trả lời: Mọi thao tác thay đổi đơn vị của quân nhân (tạo, xoá, chuyển) đều nằm trong `prisma.$transaction` cùng với câu `update so_luong = so_luong + 1` (hoặc `-1`). Nếu một thao tác lỗi, transaction rollback cả hai → bộ đếm không lệch. Ngoài ra em có script `recalcSoLuong.ts` ở DevZone để tính lại từ `COUNT(*)` thực — phòng trường hợp lỗi cũ làm lệch.

Q5. “Truy vấn nào em đánh giá là phức tạp nhất?”

Trả lời: Hàm `computeChainContext` xét chuỗi danh hiệu cá nhân. Phải duyệt lịch sử lùi từ năm hiện tại, tính (1) độ dài chuỗi CSTĐCS liên tục, (2) năm gần nhất nhận BKBQP/CSTĐTQ/BKTTCP, (3) số cờ BKBQP trong cửa sổ trượt 3 năm và 7 năm, (4) số CSTĐTQ trong cửa sổ 7 năm — tất cả để cấp đầu vào cho `checkChainEligibility`. Em chọn cách load 1 lần `DanhHieuHangNam` (thường ≤ 30 bản ghi/quân nhân) rồi xử lý ở JS thay vì 4-5 câu SQL `COUNT(*) WHERE` — vì round-trip DB tốn hơn nhiều.

Q6. “Em có dùng index gì không?”

Trả lời: PostgreSQL tự tạo B-tree index cho mọi cột PK và UNIQUE. Em đã thêm index thủ công cho các cột tra cứu thường xuyên qua migration `20260417_add_performance_indexes`: `BangDeXuat(status)`, `BangDeXuat(loai_de_xuat, nam)`, `SystemLog(createdAt)`, `DanhHieuHangNam(quan_nhan_id, nam)`, `LichSuChucVu(quan_nhan_id)`. Hai cột tổ hợp `quan_nhan_id + nam` ở `DanhHieuHangNam` là composite index đồng thời là UNIQUE.

Q7. “Có chống SQL Injection không?”

Trả lời: Có, chống mặc định ở 2 lớp: (1) **Zod validation** ở middleware kiểm tra kiểu/định dạng `req.body`, `req.query`, `req.params` — không hợp lệ thì trả 400 ngay; (2) **Prisma luôn dùng prepared statement** (parameterized query) — tham số được escape tự động, không thể inject SQL. Trường hợp `$queryRawUnsafe` (em chỉ dùng ở 1 script đổi tên cột) thì input là tên cột hard-code chứ không phải user input nên cũng an toàn.

Q8. “Có chỗ nào em phải tối ưu performance đặc biệt không?”

Trả lời: Có. Khi tính lại tổng thể (POST `/api/profile/recalc-all`) cho ~1.247 quân nhân, lần đầu em làm sequential mất ~50 giây. Sau khi: (1) batch load `DanhHieuHangNam` cho tất cả quân nhân trong 1 query (where: `{quan_nhan_id: {in: [...]}}`), (2) xử lý song song bằng `Promise.all` chia chunk 50 quân nhân, (3) batch upsert `HoSoHangNam` — thời gian giảm còn 18 giây.

10. Truy vấn nâng cao — top-N gần nhất, time window, trend

10.1 — 30 log gần nhất theo action APPROVE trong 30 ngày

Prisma

```
\NormalTok{prisma}\OperatorTok{.}\AttributeTok{systemLog}\OperatorTok{.}\FunctionTok{findMany}\NormalTok{(\{\}
\NormalTok{\ where}\OperatorTok{:}\NormalTok{\{\}
\NormalTok{\   action}\OperatorTok{:}\StringTok{\textquotesingle}\APPROVE\textquotesingle}\OperatorTok{,}
\NormalTok{\   createdAt}\OperatorTok{:}\NormalTok{\{\ gte}\OperatorTok{:}\KeywordTok{new}\BuiltInTok{Date}\NormalTok{
\NormalTok{\}\}\OperatorTok{,}
\NormalTok{\ orderBy}\OperatorTok{:}\NormalTok{\{\ createdAt}\OperatorTok{:}\StringTok{\textquotesingle}\desc\te
\NormalTok{\ take}\OperatorTok{:}\DecValTok{30}\OperatorTok{,}
\NormalTok{\ include}\OperatorTok{:}\NormalTok{\{\ TaiKhoan}\OperatorTok{:}\NormalTok{\{\ include}\OperatorTok{:}
\NormalTok{\}\}\OperatorTok{;}
```

SQL

```
\KeywordTok{SELECT}\NormalTok{\ s.}\OperatorTok{*}\NormalTok{\{ q.ho_ten }\KeywordTok{AS}\NormalTok{\ nguoi_thuc_
\KeywordTok{FROM}\NormalTok{\ SystemLog s}
\KeywordTok{LEFT}\NormalTok{\KeywordTok{JOIN}\NormalTok{\ TaiKhoan tk }\KeywordTok{ON}\NormalTok{\ s.nguoi_thuc_hien_id }
\KeywordTok{LEFT}\NormalTok{\KeywordTok{JOIN}\NormalTok{\ QuanNhan q }\KeywordTok{ON}\NormalTok{\ tk.quan_nhan_id }\C
\KeywordTok{WHERE}\NormalTok{\ s.action }\OperatorTok{=}\StringTok{\textquotesingle}\APPROVE\textquotesingle}\}
\KeywordTok{AND}\NormalTok{\ s.createdAt }\OperatorTok{\textgreater{}}=\NormalTok{\ NOW() }\OperatorTok{-}\Dat
\KeywordTok{ORDER}\NormalTok{\KeywordTok{BY}\NormalTok{\ s.createdAt }\KeywordTok{DESC}
\KeywordTok{LIMIT}\NormalTok{\DecValTok{30}\NormalTok{;}
```

10.2 — 30 đề xuất gần nhất theo trạng thái và năm

Prisma

```
\NormalTok{prisma}\OperatorTok{.}\AttributeTok{bangDeXuat}\OperatorTok{.}\FunctionTok{findMany}\NormalTok{(}\{\{
\NormalTok{  where}\OperatorTok{:}\NormalTok{\{ \{ nam}\OperatorTok{:} \DecValTok{2026}\NormalTok{\} }\OperatorTok{
\NormalTok{  orderBy}\OperatorTok{:}\NormalTok{\{ \{
\NormalTok{    \{ status}\OperatorTok{:} \StringTok{\textquotesingle}asc\textquotesingle}\NormalTok{\} }\OperatorTok{
\NormalTok{    \{ createdAt}\OperatorTok{:} \StringTok{\textquotesingle}desc\textquotesingle}\NormalTok{\} }\NormalTok{\} }\OperatorTok{
\NormalTok{  }\OperatorTok{,}\NormalTok{\}
\NormalTok{  take}\OperatorTok{:} \DecValTok{30}\OperatorTok{,}\NormalTok{\}
\NormalTok{  include}\OperatorTok{:}\NormalTok{\{ \{ NguoiDeXuat}\OperatorTok{:} \KeywordTok{true}\OperatorTok{,}\NormalTok{\} }\OperatorTok{
\NormalTok{\} }\OperatorTok{;}
```

SQL

```
\KeywordTok{SELECT}\NormalTok{\{ b.}\OperatorTok{*}
\KeywordTok{FROM}\NormalTok{\{ BangDeXuat b}
\KeywordTok{WHERE}\NormalTok{\{ b.nam }\OperatorTok{=} \DecValTok{2026}
\KeywordTok{ORDER} \KeywordTok{BY}\NormalTok{\{ b.status }\KeywordTok{ASC}\NormalTok{\{, b.createdAt }\KeywordTok{DESC}
\KeywordTok{LIMIT} \DecValTok{30}\NormalTok{;}
```

10.3 — Đề xuất kệt ở PENDING quá 7 ngày (cảnh báo SLA)

Prisma

```
\NormalTok{prisma}\OperatorTok{.}\AttributeTok{bangDeXuat}\OperatorTok{.}\FunctionTok{findMany}\NormalTok{(}\{\{
\NormalTok{  where}\OperatorTok{:}\NormalTok{\{ \{
\NormalTok{    status}\OperatorTok{:} \StringTok{\textquotesingle}PENDING\textquotesingle}\NormalTok{\} }\OperatorTok{,}\NormalTok{\}
\NormalTok{    createdAt}\OperatorTok{:}\NormalTok{\{ \{ lt}\OperatorTok{:} \KeywordTok{new} \BuiltInTok{Date}\NormalTok{\} }\OperatorTok{,}\NormalTok{\}
\NormalTok{  }\OperatorTok{,}\NormalTok{\}
\NormalTok{  orderBy}\OperatorTok{:}\NormalTok{\{ \{ createdAt}\OperatorTok{:} \StringTok{\textquotesingle}asc\textquotesingle}\NormalTok{\} }\OperatorTok{
\NormalTok{\} }\OperatorTok{;}
```

SQL

```
\KeywordTok{SELECT} \OperatorTok{*}\NormalTok{\{, NOW() }\OperatorTok{-}\NormalTok{\{ createdAt }\KeywordTok{AS}\NormalTok{\}
\KeywordTok{FROM}\NormalTok{\{ BangDeXuat}
\KeywordTok{WHERE}\NormalTok{\{ status }\OperatorTok{=} \StringTok{\textquotesingle}PENDING\textquotesingle}\NormalTok{\}
\KeywordTok{AND}\NormalTok{\{ createdAt }\OperatorTok{\textless}\NormalTok{\{ NOW() }\OperatorTok{-} \DataTypeTok{DATE}
\KeywordTok{ORDER} \KeywordTok{BY}\NormalTok{\{ createdAt }\KeywordTok{ASC}\NormalTok{;}
```

10.4 — Hoạt động hệ thống theo ngày trong 30 ngày qua (trend)

SQL (làm trên DB, hiệu quả cho trend chart)

```
\KeywordTok{SELECT} \DataTypeTok{DATE}\NormalTok{\{ (createdAt) }\KeywordTok{AS}\NormalTok{\{ ngay,}
\FunctionTok{COUNT}\NormalTok{\{ ()\OperatorTok{*}\NormalTok{\} }\KeywordTok{FILTER}\NormalTok{\{ ()\KeywordTok{
\FunctionTok{COUNT}\NormalTok{\{ ()\OperatorTok{*}\NormalTok{\} }\KeywordTok{FILTER}\NormalTok{\{ ()\KeywordTok{
\FunctionTok{COUNT}\NormalTok{\{ ()\OperatorTok{*}\NormalTok{\} }\KeywordTok{FILTER}\NormalTok{\{ ()\KeywordTok{
\FunctionTok{COUNT}\NormalTok{\{ ()\OperatorTok{*}\NormalTok{\} }\KeywordTok{FILTER}\NormalTok{\{ ()\KeywordTok{
\FunctionTok{COUNT}\NormalTok{\{ ()\OperatorTok{*}\NormalTok{\} }\KeywordTok{AS}\NormalTok{\{ tong}
\KeywordTok{FROM}\NormalTok{\{ SystemLog}
\KeywordTok{WHERE}\NormalTok{\{ createdAt }\OperatorTok{\textgreater}\NormalTok{\{ NOW() }\OperatorTok{-} \DataTypeTok{DATE}
\KeywordTok{GROUP} \KeywordTok{BY} \DataTypeTok{DATE}\NormalTok{\{ (createdAt) }
\KeywordTok{ORDER} \KeywordTok{BY}\NormalTok{\{ ngay;}
```

10.5 — 30 quân nhân sắp đủ điều kiện BKBQP trong năm tới

SQL (kết hợp 2 điều kiện: chuỗi CSTĐCS hiện tại = 1, có NCKH năm hiện tại)

```
\KeywordTok{SELECT}\NormalTok{\{ q.}\KeywordTok{id}\NormalTok{\{, q.ho\_ten, hsn.cstdcs\_lien\_tuc}
\KeywordTok{FROM}\NormalTok{\{ QuanNhan q}
\KeywordTok{JOIN}\NormalTok{\{ HoSoHangNam hsn }\KeywordTok{ON}\NormalTok{\{ hsn.quan\_nhan\_id }\OperatorTok{=}\NormalTok{\{ q.}\KeywordTok{id}
\KeywordTok{WHERE}\NormalTok{\{ hsn.cstdcs\_lien\_tuc }\OperatorTok{=} \DecValTok{1} \CommentTok{--}
\KeywordTok{AND} \KeywordTok{EXISTS}\NormalTok{\{ ()
\KeywordTok{SELECT} \DecValTok{1} \KeywordTok{FROM}\NormalTok{\{ DanhHieuHangNam d}
\KeywordTok{WHERE}\NormalTok{\{ d.quan\_nhan\_id }\OperatorTok{=}\NormalTok{\{ q.}\KeywordTok{id}
\KeywordTok{AND}\NormalTok{\{ d.nam }\OperatorTok{=} \FunctionTok{EXTRACT}\NormalTok{\{ ()\DataTypeTok{YEAR} \Ke
\KeywordTok{AND}\NormalTok{\{ d.danh\_hieu }\OperatorTok{=} \StringTok{\textquotesingle}CSTD\textquotesingle}\NormalTok{\}
\NormalTok{\} }
\KeywordTok{ORDER} \KeywordTok{BY}\NormalTok{\{ q.ho\_ten}
\KeywordTok{LIMIT} \DecValTok{30}\NormalTok{;}
```

10.6 — Đơn vị có tỷ lệ phê duyệt thấp nhất (cảnh báo)

SQL

```
\KeywordTok{SELECT}\NormalTok{ cq.ten\_don\_vi, }
  \FunctionTok{COUNT}\NormalTok{ ( )\OperatorTok{*}\NormalTok{ ( ) } \KeywordTok{AS}\NormalTok{ tong\_de\_xuat, }
  \FunctionTok{COUNT}\NormalTok{ ( )\OperatorTok{*}\NormalTok{ ( ) } \KeywordTok{FILTER}\NormalTok{ ( )\KeywordTok{ }
  \FunctionTok{ROUND}\NormalTok{ ( )
    \FloatTok{100.0} \OperatorTok{*} \FunctionTok{COUNT}\NormalTok{ ( )\OperatorTok{*}\NormalTok{ ( ) } \KeywordTok{
    \DecValTok{2}
\NormalTok{ (
  ) } \KeywordTok{AS}\NormalTok{ ti\_le\_duyet\_pct}
\KeywordTok{FROM}\NormalTok{ BangDeXuat b}
\KeywordTok{JOIN}\NormalTok{ CoQuanDonVi cq } \KeywordTok{ON}\NormalTok{ cq. } \KeywordTok{id} \OperatorTok{=}\NormalTok{
\KeywordTok{WHERE}\NormalTok{ b.nam } \OperatorTok{=}\NormalTok{ \DecValTok{2026}
\KeywordTok{GROUP} \KeywordTok{BY}\NormalTok{ cq. } \KeywordTok{id}\NormalTok{ , cq.ten\_don\_vi}
\KeywordTok{HAVING} \FunctionTok{COUNT}\NormalTok{ ( )\OperatorTok{*}\NormalTok{ ( ) } \OperatorTok{\textgreater} \NormalTok{ ( ) } \KeywordTok{ASC}
\KeywordTok{ORDER} \KeywordTok{BY}\NormalTok{ ti\_le\_duyet\_pct } \KeywordTok{ASC}
\KeywordTok{LIMIT} \DecValTok{10}\NormalTok{ ; }
```

10.7 — So sánh số khen thưởng năm hiện tại vs năm trước (year-over-year)

SQL

```
\KeywordTok{WITH}\NormalTok{ this\_year } \KeywordTok{AS}\NormalTok{ (
  \KeywordTok{SELECT} \FunctionTok{COUNT}\NormalTok{ ( )\OperatorTok{*}\NormalTok{ ( ) } \KeywordTok{AS}\NormalTok{ sl
\NormalTok{ , ) }
\NormalTok{ last\_year } \KeywordTok{AS}\NormalTok{ (
  \KeywordTok{SELECT} \FunctionTok{COUNT}\NormalTok{ ( )\OperatorTok{*}\NormalTok{ ( ) } \KeywordTok{AS}\NormalTok{ sl
\NormalTok{ ) }
\KeywordTok{SELECT}\NormalTok{ this\_year.sl } \KeywordTok{AS}\NormalTok{ nam\_2026, }
\NormalTok{ last\_year.sl } \KeywordTok{AS}\NormalTok{ nam\_2025, }
\NormalTok{ this\_year.sl } \OperatorTok{-}\NormalTok{ last\_year.sl } \KeywordTok{AS}\NormalTok{ chenh\_le
  \FunctionTok{ROUND}\NormalTok{ ( )\FloatTok{100.0} \OperatorTok{*}\NormalTok{ ( this\_year.sl )\OperatorTok{
\KeywordTok{FROM}\NormalTok{ this\_year, last\_year ; }
```

10.8 — Người dùng hoạt động nhiều nhất 7 ngày qua

SQL

```
\KeywordTok{SELECT}\NormalTok{ tk.username, q.ho\_ten, } \FunctionTok{COUNT}\NormalTok{ ( )\OperatorTok{*}\NormalTok{ (
  \FunctionTok{MAX}\NormalTok{ ( s.createdAt ) } \KeywordTok{AS}\NormalTok{ lan\_cuoi}
\KeywordTok{FROM}\NormalTok{ SystemLog s}
\KeywordTok{JOIN}\NormalTok{ TaiKhoan tk } \KeywordTok{ON}\NormalTok{ s.nguoi\_thuc\_hien\_id } \OperatorTok{=}\NormalTok{
\KeywordTok{JOIN}\NormalTok{ QuanNhan q } \KeywordTok{ON}\NormalTok{ tk.quan\_nhan\_id } \OperatorTok{=}\NormalTok{
\KeywordTok{WHERE}\NormalTok{ s.createdAt } \OperatorTok{\textgreater} \NormalTok{ NOW() } \OperatorTok{-}\NormalTok{ \Date
\KeywordTok{GROUP} \KeywordTok{BY}\NormalTok{ tk. } \KeywordTok{id}\NormalTok{ , tk.username, q.ho\_ten}
\KeywordTok{ORDER} \KeywordTok{BY}\NormalTok{ so\_thao\_tac } \KeywordTok{DESC}
\KeywordTok{LIMIT} \DecValTok{20}\NormalTok{ ; }
```

10.9 — Quân nhân có nhiều bản ghi nhất bị thay đổi (đáng nghi)

SQL (audit/security review)

```
\KeywordTok{SELECT}\NormalTok{ s.tai\_nguyen\_id } \KeywordTok{AS}\NormalTok{ quan\_nhan\_id, q.ho\_ten, } \FunctionTok{COUNT}\NormalTok{ ( )\OperatorTok{*}\NormalTok{ (
\KeywordTok{FROM}\NormalTok{ SystemLog s}
\KeywordTok{LEFT} \KeywordTok{JOIN}\NormalTok{ QuanNhan q } \KeywordTok{ON}\NormalTok{ q. } \KeywordTok{id} \OperatorTok{=}\NormalTok{
\KeywordTok{WHERE}\NormalTok{ s. } \KeywordTok{resource} \OperatorTok{=}\NormalTok{ \StringTok{\textquotesingle}personnel\textquotesingle}
  \KeywordTok{AND}\NormalTok{ s.action } \KeywordTok{IN}\NormalTok{ ( )\StringTok{\textquotesingle}UPDATE\textquotesingle}
  \KeywordTok{AND}\NormalTok{ s.createdAt } \OperatorTok{\textgreater} \NormalTok{ NOW() } \OperatorTok{-}\NormalTok{ \Date
\KeywordTok{GROUP} \KeywordTok{BY}\NormalTok{ s.tai\_nguyen\_id, q.ho\_ten}
\KeywordTok{HAVING} \FunctionTok{COUNT}\NormalTok{ ( )\OperatorTok{*}\NormalTok{ ( ) } \OperatorTok{\textgreater} \NormalTok{ ( ) } \KeywordTok{ASC}
\KeywordTok{ORDER} \KeywordTok{BY}\NormalTok{ so\_lan\_thay\_doi } \KeywordTok{DESC}\NormalTok{ ; }
```

11. Xử lý đồng thời, race condition và transaction

11.1 Tình huống race condition trong PM QLKT

Vấn đề: Hai Admin A và B cùng mở chi tiết một đề xuất PENDING. A bấm “Phê duyệt” trước, sau đó B cũng bấm “Phê duyệt” mà không refresh — kết quả mong muốn: B phải nhận lỗi 409, KHÔNG được duyệt lần 2.

Giải pháp 1 — Optimistic locking dùng version field:

Thêm cột version vào BangDeXuat:

```
\KeywordTok{ALTER} \KeywordTok{TABLE}\NormalTok{ BangDeXuat } \KeywordTok{ADD} \KeywordTok{COLUMN}\NormalTok{ vers
```

Khi update phê duyệt:

```
\KeywordTok{const}\NormalTok{ updated }\OperatorTok{=} \ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\NormalTok{ where}\OperatorTok{:}\NormalTok{ \{ id}\OperatorTok{:}\NormalTok{ proposalId}\OperatorTok{,}\NormalTok{ data}\OperatorTok{:}\NormalTok{ \{ status}\OperatorTok{:} \StringTok{\textquotesingle}APPROVED\textquotesingle}\NormalTok{ }\}\OperatorTok{;}\ControlFlowTok{if}\NormalTok{ (updated)\OperatorTok{.}\AttributeTok{count} }\OperatorTok{===} \DecValTok{0}\NormalTok{ }\ControlFlowTok{throw} \KeywordTok{new} \FunctionTok{ConflictError}\NormalTok{ (}\StringTok{\textquotesingle}Đề xuất đã được duyệt\textquotesingle}\NormalTok{ )}\OperatorTok{;}
```

Giải thích: updateMany trả về count = 0 nếu WHERE không khớp (nghĩa là version đã thay đổi do A duyệt trước). Đây là cách kiểm tra “compare-and-swap” trên DB — atomic, không cần lock pessimistic.

Giải pháp 2 — Pessimistic lock (SELECT FOR UPDATE):

Dùng raw query trong transaction:

```
\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\FunctionTok{$transaction}\NormalTok{ (}\KeywordTok{async}\NormalTok{ () }\KeywordTok{const}\NormalTok{ [proposal] }\OperatorTok{=} \ControlFlowTok{await}\NormalTok{ tx}\OperatorTok{.}\VerbatimStringTok{ SELECT * FROM BangDeXuat WHERE id = }\SpecialCharTok{$\}\NormalTok{ proposalId}\SpecialCharTok{\textasciigrave{}}\OperatorTok{;}\ControlFlowTok{if}\NormalTok{ (proposal)\OperatorTok{.}\AttributeTok{status} }\OperatorTok{!==} \StringTok{\textquotesingle}Đã duyệt\textquotesingle}\ControlFlowTok{throw} \KeywordTok{new} \FunctionTok{ConflictError}\NormalTok{ (}\StringTok{\textquotesingle}Đề xuất đã được duyệt\textquotesingle}\NormalTok{ )}\ControlFlowTok{await}\NormalTok{ tx}\OperatorTok{.}\AttributeTok{bangDeXuat}\OperatorTok{.}\FunctionTok{update}\NormalTok{ (}\NormalTok{ }\OperatorTok{;}
```

Khi nào dùng cái nào? - Optimistic (em đang dùng) khi conflict hiếm — hiệu năng tốt vì không lock. **- Pessimistic** (FOR UPDATE) khi conflict nhiều và logic dài — đảm bảo tuyệt đối nhưng giảm throughput.

11.2 Race condition khi cập nhật so_luong đơn vị

Vấn đề: 2 admin cùng tạo quân nhân vào cùng 1 đơn vị tại cùng thời điểm. Nếu code làm:

```
\CommentTok{// SAI - race condition!}\KeywordTok{const}\NormalTok{ dv }\OperatorTok{=} \ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\AttributeTok{coQuanDonVi}\OperatorTok{.}\FunctionTok{update}\NormalTok{ (}\NormalTok{ }\OperatorTok{;}
```

Hai luồng đọc cùng so_luong = 10, mỗi luồng cộng 1 ghi 11 → kết quả 11 thay vì 12.

Giải pháp: dùng atomic increment của Prisma (sinh ra SET so_luong = so_luong + 1 ở DB):

```
\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\AttributeTok{coQuanDonVi}\OperatorTok{.}\FunctionTok{update}\NormalTok{ (}\NormalTok{ where}\OperatorTok{:}\NormalTok{ \{ id }\}\OperatorTok{,}\NormalTok{ data}\OperatorTok{:}\NormalTok{ \{ so_luong }\}\OperatorTok{,}\NormalTok{ \{ increment }\}\OperatorTok{;}
```

PostgreSQL serialise mọi UPDATE trên cùng row — không thể xen ngang.

11.3 Transaction lồng + truyền tx qua nhiều Repository

Pattern em dùng:

```

\CommentTok{// Repository nhận tham số tx tùy chọn}
\KeywordTok{async} \KeywordTok{function} \FunctionTok{findByPersonnelId}\NormalTok{({}
\NormalTok{ personnelId}\OperatorTok{:}) \DataTypeTok{string}\OperatorTok{,}
\NormalTok{ tx}\OperatorTok{:}\NormalTok{ PrismaLike }\OperatorTok{=}\NormalTok{ prisma}\OperatorTok{,} \Comm
\NormalTok{() }\}
\ControlFlowTok{return}\NormalTok{ tx}\OperatorTok{.}\AttributeTok{danhHieuHangNam}\OperatorTok{.}\FunctionTok{
\NormalTok{}}\}

\CommentTok{// Service mở transaction rồi truyền tx}
\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\FunctionTok{$transaction}\NormalTok{({}\KeywordTok{async}
\KeywordTok{const}\NormalTok{ records }\OperatorTok{=}\ControlFlowTok{await}\NormalTok{ danhHieuRepo}\OperatorTok{
\ControlFlowTok{await}\NormalTok{ hoSoRepo}\OperatorTok{.}\FunctionTok{upsert}\NormalTok{(personnelId)\OperatorTok{
\ControlFlowTok{await}\NormalTok{ systemLogRepo}\OperatorTok{.}\FunctionTok{create}\NormalTok{(logData)\OperatorTok{
\NormalTok{}}}\OperatorTok{;}}

```

Lý do: Tất cả query trong `$transaction` chia sẻ cùng connection và transaction context. Nếu bất kỳ câu nào throw → rollback hết. Nếu repo không nhận tx mà tự `prisma.x.findMany(...)`, query đó sẽ chạy ngoài transaction → không rollback được.

11.4 Promise.all song song — cẩn thận khi 1 cái lỗi

Vấn đề kinh điển:

```

\CommentTok{// MISTAKE: 1 lỗi {-}\textgreater{} 4 cái còn lại đã chạy nhưng kết quả mất}
\ControlFlowTok{await} \BuiltInTok{Promise}\OperatorTok{.}\FunctionTok{all}\NormalTok{({}
\FunctionTok{recalcAnnualProfile}\NormalTok{(qn1)\OperatorTok{,}
\FunctionTok{recalcAnnualProfile}\NormalTok{(qn2)\OperatorTok{,}
\FunctionTok{recalcAnnualProfile}\NormalTok{(qn3)\OperatorTok{,}
\FunctionTok{recalcAnnualProfile}\NormalTok{(qn4)\OperatorTok{,}
\FunctionTok{recalcAnnualProfile}\NormalTok{(qn5)\OperatorTok{,} \CommentTok{// throw - Promise.all reject ngay}
\NormalTok{[]}\OperatorTok{;}}

```

`Promise.all` reject ngay khi promise đầu tiên fail; các promise kia VẪN đang chạy, kết quả chúng tạo ra (vd: ghi DB thành công) sẽ KHÔNG bị rollback nếu chúng tự mở transaction riêng.

Giải pháp 1 — bao toàn bộ trong 1 `$transaction` ngoài cùng:

```

\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\FunctionTok{$transaction}\NormalTok{({}\KeywordTok{async}
\ControlFlowTok{await} \BuiltInTok{Promise}\OperatorTok{.}\FunctionTok{all}\NormalTok{(personnelIds)\OperatorTok{
\CommentTok{// nếu 1 cái throw → tx tự rollback hết}
\NormalTok{}}}\OperatorTok{;}}

```

Vì tất cả share cùng tx, khi rollback thì các thao tác đã ghi cũng được hoàn tác.

Giải pháp 2 — `Promise.allSettled` để biết cái nào fail:

```

\KeywordTok{const}\NormalTok{ results }\OperatorTok{=}\ControlFlowTok{await} \BuiltInTok{Promise}\OperatorTok{.}\FunctionTok{allSettled}\NormalTok{({}
\NormalTok{ personnelIds}\OperatorTok{.}\FunctionTok{map}\NormalTok{(id)\KeywordTok{=>}\textgreater{} \FunctionTok{
\NormalTok{()}\OperatorTok{;}}
\KeywordTok{const}\NormalTok{ failed }\OperatorTok{=}\NormalTok{ results}
\OperatorTok{.}\FunctionTok{map}\NormalTok{((r)\OperatorTok{,}\NormalTok{ i})\KeywordTok{=>}\textgreater{} \NormalTok{
\OperatorTok{.}\FunctionTok{filter}\NormalTok{({}\BuiltInTok{Boolean}\NormalTok{()}\OperatorTok{;}}
\ControlFlowTok{if}\NormalTok{(failed)\OperatorTok{.}\AttributeTok{length}\OperatorTok{>}\textgreater{} \DecValTok{0}
\ControlFlowTok{await} \FunctionTok{writeSystemLog}\NormalTok{({}\{ action}\OperatorTok{:}\StringTok{"\textquotesingle}
\NormalTok{}}\}

```

Khi nào dùng cái nào? - Recalc trong cùng đề xuất duyệt → **giải pháp 1** (cần atomic). - Recalc batch hằng ngày → **giải pháp 2** (chấp nhận 1 vài cái fail, ghi log để xử lý sau).

11.5 Bulk import 500 dòng, dòng 250 lỗi → toàn bộ rollback

Cài đặt (`services/proposal/approve/import.ts`):

```

\KeywordTok{async} \KeywordTok{function} \FunctionTok{confirmImport}\NormalTok{(rows)\OperatorTok{:}\NormalTok{ I
\ControlFlowTok{return}\NormalTok{ prisma}\OperatorTok{.}\FunctionTok{$transaction}\NormalTok{({}\KeywordTok{asy
\ControlFlowTok{for}\NormalTok{({}\KeywordTok{const}\NormalTok{ [idx]\OperatorTok{,}\NormalTok{ row} }\Keywor

```

```

\ControlFlowTok{try}\NormalTok{ \{\}
\ControlFlowTok{await}\NormalTok{ tx}\OperatorTok{.}\AttributeTok{danhHieuHangNam}\OperatorTok{.}\FunctionTok{
\NormalTok{ \} }\ControlFlowTok{catch}\NormalTok{ (e) \{\}
\ControlFlowTok{throw} \KeywordTok{new} \FunctionTok{ImportError}\NormalTok{ { }\VerbatimStringTok{\textasciit
\NormalTok{ \}\}
\NormalTok{ \}\}
\NormalTok{ \}\}\OperatorTok{,}\NormalTok{ \{\}
\NormalTok{ timeout}\OperatorTok{:} \DecValTok{60\_000}\OperatorTok{,} \CommentTok{// 60s – bulk lớn
\NormalTok{ isolationLevel}\OperatorTok{:} \StringTok{\textquotesingle}Serializable\textquotesingle}\Operator
\NormalTok{ \}\}\OperatorTok{;}
\NormalTok{ \}\}

```

Quan sát hành vi: - Dòng 1-249 ghi thành công vào WAL của PostgreSQL. - Dòng 250 throw → Prisma rollback transaction. - PostgreSQL undo các thay đổi trong WAL → DB trở về trạng thái trước. - API trả 400 với chỉ số dòng lỗi cho user.

Lý do tách 2 bước “preview” + “confirm”: - Bước **preview** chỉ đọc & validate, không mở transaction → cho phép user fix file Excel trước khi tốn thời gian transaction dài. - Bước **confirm** mở transaction một lần với toàn bộ dòng đã valid → giảm rủi ro lỗi giữa chừng.

11.6 Deadlock và cách phòng tránh

Tình huống: Hai transaction T1 và T2 đều cập nhật QuanNhan A và QuanNhan B. - T1: lock A, đợi B - T2: lock B, đợi A → deadlock. PostgreSQL phát hiện sau ~1s và abort 1 transaction.

Cách phòng: 1. **Luôn lock theo cùng thứ tự** (vd: theo ID tăng dần). Trong transaction phê duyệt đề xuất, em sort `personnelIds.sort()` trước khi loop để mọi transaction đều dùng các row theo cùng order. 2. **Tránh transaction quá dài** — chỉ những thao tác cần atomic mới đặt trong `$transaction`. Việc gửi email/socket.io đặt sau khi commit. 3. **Set timeout hợp lý:** `prisma.$transaction(fn, {timeout: 10_000})` — nếu transaction kéo dài bất thường thì abort sớm thay vì giữ lock vô hạn.

11.7 Idempotency — phê duyệt nhiều lần cùng 1 đề xuất

Vấn đề: Mạng chậm, FE gửi 2 request POST `/api/proposals/:id/approve` liên tiếp. Cần đảm bảo chỉ duyệt 1 lần.

Giải pháp: Backend kiểm tra `status === 'PENDING'` ngay đầu transaction:

```

\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\FunctionTok{$transaction}\NormalTok{ { }\KeywordTok{async}
\KeywordTok{const}\NormalTok{ proposal }\OperatorTok{=}\ControlFlowTok{await}\NormalTok{ tx}\OperatorTok{.}\At
\ControlFlowTok{if}\NormalTok{ { }\OperatorTok{!}\NormalTok{ {proposal} }\ControlFlowTok{throw} \KeywordTok{new} \
\ControlFlowTok{if}\NormalTok{ {proposal}\OperatorTok{.}\AttributeTok{status} \OperatorTok{!==} \StringTok{\textquotesingle}tex
\ControlFlowTok{throw} \KeywordTok{new} \FunctionTok{ConflictError}\NormalTok{ { }\StringTok{\textquotesingle}
\NormalTok{ \}\}
\CommentTok{// ... duyệt logic}
\NormalTok{ \}\}\OperatorTok{;}

```

Request thứ 2 thấy `status = 'APPROVED'` → trả 409, không duyệt lần 2.

11.8 Connection pool — backend 1 instance, ~50 user đồng thời

Cấu hình (Prisma đọc từ `DATABASE_URL`):

`DATABASE_URL=postgresql://user:pass@localhost:5432/qlkt?connection_limit=10&pool_t`

- `connection_limit=10` — pool tối đa 10 connection. Đủ cho ~50 user vì mỗi request HTTP thường giải phóng connection sau ~100ms.
- `pool_timeout=20` — request thứ 11 đợi tối đa 20s nếu pool hết connection.

Khi nào tăng pool? Khi log thấy nhiều `pool_timeout error` hoặc latency p95 cao hơn p50 nhiều — biểu hiện đang queue chờ connection.

12. Câu hỏi giám khảo nâng cao + đáp án (kèm code)

Q9. “Hai Admin cùng phê duyệt một đề xuất ở cùng thời điểm — chuyện gì xảy ra?”

Trả lời: Em xử lý bằng compare-and-swap (CAS): điều kiện WHERE chứa cả `status = 'PENDING'` nên chỉ 1 trong 2 request thành công.

Prisma

```
\KeywordTok{const}\NormalTok{ updated }\OperatorTok{=}\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\NormalTok{
}\NormalTok{ where}\OperatorTok{:}\NormalTok{ \{ id}\OperatorTok{:}\NormalTok{ proposalId}\OperatorTok{,}\NormalTok{
}\NormalTok{ data}\OperatorTok{:}\NormalTok{ }\NormalTok{ status}\OperatorTok{:}\NormalTok{ \StringTok{\textquotesingle}APPROVED\textquotesingle}
}\NormalTok{ }\OperatorTok{;}\NormalTok{
}\ControlFlowTok{if}\NormalTok{ (updated)\OperatorTok{.}\AttributeTok{count}\OperatorTok{===}\DecValTok{0}\NormalTok{
}\ControlFlowTok{throw}\NormalTok{ new}\NormalTok{ }\FunctionTok{ConflictError}\NormalTok{ (\StringTok{\textquotesingle}Dé
}\NormalTok{ )}
```

SQL

```
\CommentTok{{-}{-} Admin A chạy trước:}
\KeywordTok{UPDATE}\NormalTok{ BangDeXuat }\KeywordTok{SET}\NormalTok{ status }\OperatorTok{=} \StringTok{\texttt{}}
\KeywordTok{WHERE} \KeywordTok{id} \OperatorTok{=}\NormalTok{ \$ }\DecValTok{1} \KeywordTok{AND}\NormalTok{ status }
\CommentTok{{-}{-} Postgres trả: UPDATE 1}

\CommentTok{{-}{-} Admin B chạy sau (status đã là APPROVED):}
\KeywordTok{UPDATE}\NormalTok{ BangDeXuat }\KeywordTok{SET}\NormalTok{ status }\OperatorTok{=} \StringTok{\texttt{}}
\KeywordTok{WHERE} \KeywordTok{id} \OperatorTok{=}\NormalTok{ \$ }\DecValTok{1} \KeywordTok{AND}\NormalTok{ status }
\CommentTok{{-}{-} Postgres trả: UPDATE 0 → backend throw 409}
```

PostgreSQL serialise mọi UPDATE trên cùng row qua row-level lock, nên không có cảnh “cả 2 cùng update”.

Q10. “Khi nhập 500 dòng Excel mà dòng 250 lỗi, dữ liệu 249 dòng đầu đã ghi thì sao?”

Trả lời: Tất cả 500 dòng nằm trong 1 \$transaction. Khi dòng 250 throw, Prisma gửi ROLLBACK → 249 dòng trước cũng bị undo.

Prisma (services/proposal/approve/import.ts)

```

\KeywordTok{async} \KeywordTok{function} \FunctionTok{confirmImport}\NormalTok{(rows)\OperatorTok{:}\NormalTok{( I
\ControlFlowTok{return}\NormalTok{( prisma)\OperatorTok{.}\FunctionTok{$transaction}\NormalTok{( ()\KeywordTok{asy
\ControlFlowTok{for}\NormalTok{( ()\KeywordTok{let}\NormalTok{( i )\OperatorTok{=}\DecValTok{0}\OperatorTok{;}
\ControlFlowTok{try}\NormalTok{( \{\}
\ControlFlowTok{await}\NormalTok{( tx)\OperatorTok{.}\AttributeTok{danhHieuHangNam}\OperatorTok{.}\FunctionTok{
\NormalTok{( \} )}\ControlFlowTok{catch}\NormalTok{( e )\{\}
\CommentTok{// Throw {-}\textgreater{} Prisma tự gửi ROLLBACK xuống Postgres}
\ControlFlowTok{throw} \KeywordTok{new} \FunctionTok{ImportError}\NormalTok{( ()\VerbatimStringTok{\textasc
\NormalTok{( \} )}
\NormalTok{( \} )}
\NormalTok{( \} )}\OperatorTok{,}\NormalTok{( \{ timeout}\OperatorTok{:}\DecValTok{60\_000}\NormalTok{( \} )}\OperatorTok{
\NormalTok{( \} )}

```

SQL trong transaction

```

\ControlFlowTok{BEGIN}\NormalTok{;;
\KeywordTok{INSERT} \KeywordTok{INTO}\NormalTok{ { DanhHieuHangNam ()\OperatorTok{..}\NormalTok{(.) } \KeywordTok{VALUES}\NormalTok{ {
\KeywordTok{INSERT} \KeywordTok{INTO}\NormalTok{ { DanhHieuHangNam ()\OperatorTok{..}\NormalTok{(.) } \KeywordTok{VALUES}\NormalTok{ {
\OperatorTok{..}\NormalTok{(.) }
\KeywordTok{INSERT} \KeywordTok{INTO}\NormalTok{ { DanhHieuHangNam ()\OperatorTok{..}\NormalTok{(.) } \KeywordTok{VALUES}\NormalTok{ {
\KeywordTok{INSERT} \KeywordTok{INTO}\NormalTok{ { DanhHieuHangNam ()\OperatorTok{..}\NormalTok{(.) } \KeywordTok{VALUES}\NormalTok{ {
\KeywordTok{ROLLBACK}\NormalTok{;; \CommentTok{-}} Postgres undo 249 dòng từ WAL}

```

Test verify (tests/import/personalAnnualImport.test.ts):

```
\FunctionTok{it}\NormalTok{({}\StringTok{\textquotesingle}{rollback toàn bộ khi dòng giữa lỗi\textquotesingle}{})}\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\FunctionTok{$transaction}\NormalTok{({}\KeywordTok{async}\ControlFlowTok{await}\BuiltInTok{Promise}\OperatorTok{.}\FunctionTok{all}\NormalTok{({}\ControlFlowTok{await}\FunctionTok{expect}\NormalTok{({}\FunctionTok{confirmImport}\NormalTok{(rowsWithError250}\KeywordTok{const}\NormalTok{ after }\OperatorTok{=}\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\FunctionTok{expect}\NormalTok{(after)}\OperatorTok{.}\FunctionTok{toBe}\NormalTok{(before)}\OperatorTok{;}\NormalTok{)})}\OperatorTok{;})}
```

Q11. “Em chạy `Promise.all` song song 50 quân nhân để recalc — 1 cái lỗi thì sao?”

Trả lời: 2 mode tùy ngữ cảnh.

Mode A — cần atomic (recalc trong cùng phê duyệt):

```
\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\FunctionTok{$transaction}\NormalTok{({}\KeywordTok{async}\ControlFlowTok{await}\BuiltInTok{Promise}\OperatorTok{.}\FunctionTok{all}\NormalTok{({}\NormalTok{ personnelIds}\OperatorTok{.}\FunctionTok{map}\NormalTok{(id }\KeywordTok{=}\textgreater{} }\FunctionTok{throw}\NormalTok{({}\NormalTok{ })}\OperatorTok{;})\CommentTok{// Nếu 1 promise throw, tx tự rollback toàn bộ}\NormalTok{)})}\OperatorTok{;}
```

Tất cả 50 promise dùng chung tx → fail 1 = rollback 50.

Mode B — chấp nhận lỗi cục bộ (recalc batch hằng ngày):

```
\KeywordTok{const}\NormalTok{ results }\OperatorTok{=}\ControlFlowTok{await}\BuiltInTok{Promise}\OperatorTok{.}\FunctionTok{map}\NormalTok{(id }\KeywordTok{=}\textgreater{} }\FunctionTok{throw}\NormalTok{({}\NormalTok{ failed }\OperatorTok{=}\NormalTok{ results}\OperatorTok{.}\FunctionTok{map}\NormalTok{(r)\OperatorTok{,}\NormalTok{ i }\KeywordTok{=}\textgreater{} }\NormalTok{ failed }\OperatorTok{.}\FunctionTok{filter}\NormalTok{({}\BuiltInTok{Boolean}\NormalTok{)})}\OperatorTok{;}\ControlFlowTok{if}\NormalTok{ (failed)\OperatorTok{.}\AttributeTok{length}\OperatorTok{\textgreater{} }\DecValTok{50}\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\AttributeTok{systemLog}\OperatorTok{.}\FunctionTok{create}\NormalTok{(data)\OperatorTok{:}\NormalTok{ \{ action}\OperatorTok{:}\StringTok{\textquotesingle}{RECALC\_FAILED}\NormalTok{ \}}}\OperatorTok{;}\NormalTok{ \}}\OperatorTok{;}
```

Anti-pattern phải tránh:

```
\CommentTok{// SAI: Promise.all không bao trong $transaction}\CommentTok{// 1 cái fail {-}\textgreater{} 49 cái thành công vẫn commit độc lập}\ControlFlowTok{await}\BuiltInTok{Promise}\OperatorTok{.}\FunctionTok{all}\NormalTok{({}\NormalTok{ personnelIds}\OperatorTok{.}\FunctionTok{map}\NormalTok{(id }\KeywordTok{=}\textgreater{} }\FunctionTok{throw}\NormalTok{({}\NormalTok{)})}\OperatorTok{;}
```

Q12. “Em có dùng `SELECT FOR UPDATE` bao giờ không?”

Trả lời: Hiện chưa cần (CAS đủ dùng), nhưng nếu cần lock pessimistic cho flow đọc-tính-ghi dài, em sẽ làm như sau:

Prisma raw query (Prisma 5 chưa có `forUpdate` API)

```
\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\FunctionTok{$transaction}\NormalTok{({}\KeywordTok{async}\CommentTok{// Lock row đề xuất, các transaction khác phải đợi}\KeywordTok{const}\NormalTok{ [proposal] }\OperatorTok{=}\ControlFlowTok{await}\NormalTok{ tx}\OperatorTok{.}\VerbatimStringTok{SELECT * FROM "BangDeXuat" WHERE id = }\SpecialCharTok{$}\NormalTok{(proposalId)\SpecialCharTok{$}}\ControlFlowTok{if}\NormalTok{ (proposal)\OperatorTok{.}\AttributeTok{status}\OperatorTok{!==}\StringTok{\textquotesingle}{COMPLETED}\ControlFlowTok{throw}\KeywordTok{new}\FunctionTok{ConflictError}\NormalTok{({}\StringTok{\textquotesingle}{Row is locked}\NormalTok{)})}\CommentTok{// Tính toán phức tạp \textasciitilde{}3 giây ở đây, vẫn an toàn vì row đã lock}\KeywordTok{const}\NormalTok{ computed }\OperatorTok{=}\ControlFlowTok{await}\FunctionTok{heavyEligibilityCheck}\NormalTok{ tx}\OperatorTok{.}\AttributeTok{bangDeXuat}\OperatorTok{.}\FunctionTok{update}\NormalTok{(where)\OperatorTok{:}\NormalTok{ \{ id}\OperatorTok{:}\NormalTok{ proposalId }\}}\OperatorTok{,}\NormalTok{ data}\OperatorTok{:}\NormalTok{ \{ status}\OperatorTok{:}\StringTok{\textquotesingle}{APPROVED}\NormalTok{ \}}}\OperatorTok{;}\NormalTok{ \}}\OperatorTok{;}
```


SQL tương đương

```
\ControlFlowTok{BEGIN}\NormalTok{;;}  
\KeywordTok{SELECT} \OperatorTok{*} \KeywordTok{FROM} \OtherTok{"BangDeXuat"} \KeywordTok{WHERE} \KeywordTok{id}  
\CommentTok{{-}{-} ... computation ...}  
\KeywordTok{UPDATE} \OtherTok{"BangDeXuat"} \KeywordTok{SET}\NormalTok{ status }\OperatorTok{=}\StringTok{\text}  
\KeywordTok{COMMIT}\NormalTok{;;}
```

Khi nào dùng FOR UPDATE thay vì CAS: khi đọc nhiều cột để tính rồi mới update — CAS chỉ tốt khi check 1 cò (status=PENDING).

Q13. “Deadlock có xảy ra không và em chống thế nào?”

Trả lời: Em chống bằng 2 kỹ thuật.

Kỹ thuật 1 — Lock theo thứ tự nhất quán:

```
\CommentTok{// SAI: thứ tự update phụ thuộc thứ tự input {-}\textgreater{} dễ deadlock}  
\ControlFlowTok{await} \BuiltInTok{Promise}\OperatorTok{.}\FunctionTok{all}\NormalTok{( personnelIds)\OperatorTok{}  
  
\CommentTok{// ĐÚNG: sort trước {-}\textgreater{} mọi transaction đùng row theo cùng order}  
\KeywordTok{const}\NormalTok{ sortedIds }\OperatorTok{=}\NormalTok{ []}\OperatorTok{...}\NormalTok{ personnelIds}\OperatorTok{}  
\ControlFlowTok{for}\NormalTok{ ( )}\KeywordTok{const}\NormalTok{ id }\KeywordTok{of}\NormalTok{ sortedIds }\OperatorTok{ {  
  \ControlFlowTok{await} \FunctionTok{updatePersonnel}\NormalTok{ (tx)\OperatorTok{,}\NormalTok{ id }\OperatorTok{ {  
\NormalTok{ } } }
```

Kỹ thuật 2 — Timeout transaction:

```
\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\FunctionTok{$transaction}\NormalTok{ ( )}\KeywordTok{async}  
  \CommentTok{// ... heavy work ...}  
\NormalTok{ } }\OperatorTok{,}\NormalTok{ } }  
\NormalTok{ timeout }\OperatorTok{:} \DecValTok{10\_000}\OperatorTok{,} \CommentTok{// 10s – abort nếu kéo  
\NormalTok{ maxWait }\OperatorTok{:} \DecValTok{5\_000}\OperatorTok{,} \CommentTok{// 5s – đợi lock tối d  
\NormalTok{ } }\OperatorTok{;}
```

Khi Postgres detect deadlock (~1s sau khi cycle):

```
\NormalTok{ERROR:  deadlock detected}  
\NormalTok{DETAIL:  Process }\DecValTok{12345}\NormalTok{ waits }\ControlFlowTok{for}\NormalTok{ ShareLock }\Keyw  
\NormalTok{ blocked }\KeywordTok{by}\NormalTok{ process }\FloatTok{23456.}  
\NormalTok{HINT:     See server }\FunctionTok{log}\ControlFlowTok{for}\KeywordTok{query}\NormalTok{ details.}
```

Postgres tự huỷ 1 transaction (transaction “victim”) và return lỗi 40P01. Backend bắt và retry tối đa 3 lần với exponential backoff.

Q14. “Tại sao em chia bước Excel thành preview + confirm?”

Trả lời: Để giảm thời gian giữ transaction lock.

Bước preview — chỉ đọc, KHÔNG mở transaction:

```
\CommentTok{// services/proposal/strategies/caNhanHangNamStrategy.ts}  
\KeywordTok{async} \FunctionTok{previewImport}\NormalTok{ (rows)\OperatorTok{:}\NormalTok{ ImportRow[] }\OperatorTok{ {  
  \KeywordTok{const}\NormalTok{ valid }\OperatorTok{:}\NormalTok{ ImportRow[] }\OperatorTok{=}\NormalTok{ []}\OperatorTok{  
  \KeywordTok{const}\NormalTok{ errors }\OperatorTok{:}\NormalTok{ ErrorRow[] }\OperatorTok{=}\NormalTok{ []}\OperatorTok{  
  \ControlFlowTok{for}\NormalTok{ ( )}\KeywordTok{let}\NormalTok{ i }\OperatorTok{=}\DecValTok{0}\OperatorTok{ {  
    \KeywordTok{const}\NormalTok{ result }\OperatorTok{=}\NormalTok{ schema}\OperatorTok{.}\FunctionTok{validate}  
    \ControlFlowTok{if}\NormalTok{ (result)\OperatorTok{.}\AttributeTok{error}\NormalTok{ {  
\NormalTok{ errors }\OperatorTok{.}\FunctionTok{push}\NormalTok{ (row)\OperatorTok{ {  
\NormalTok{ } }\ControlFlowTok{else}\NormalTok{ {  
  \CommentTok{// Read{-}only check: CCCD tồn tại?  
  \KeywordTok{const}\NormalTok{ exists }\OperatorTok{=}\ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{  
\NormalTok{ where }\OperatorTok{:}\NormalTok{ cccd }\OperatorTok{:}\NormalTok{ rows[i] }\OperatorTok{.}\At  
\NormalTok{ select }\OperatorTok{:}\NormalTok{ id }\OperatorTok{:}\KeywordTok{true}\NormalTok{ } }\OperatorTok{ {  
\NormalTok{ } }\OperatorTok{; }  
  \ControlFlowTok{if}\NormalTok{ ( )}\OperatorTok{!}\NormalTok{ exists }\OperatorTok{.}\FunctionTok{push}  
  \ControlFlowTok{else}\NormalTok{ valid }\OperatorTok{.}\FunctionTok{push}\NormalTok{ (rows[i] )}\OperatorTok{ {
```

Bước confirm — mở transaction CHỈ với dòng đã valid:

Lợi ích do được: với 500 dòng + 50 dòng lỗi, nếu làm 1 bước thì transaction giữ lock 30s → rollback toàn bộ → user phải sửa rồi chạy lại 30s nữa. Tách 2 bước: preview 5s không lock, sau khi sửa file confirm chỉ 3s với 450 dòng valid.

Trả lời: Mọi endpoint mutate kiểm state đầu transaction.

SQL

Detect replay từ log:

Q16. “Em có dùng index không, làm sao biết khi nào cần thêm?”

Trả lời: Có, đã thêm trong migration 20260417 add performance indexes.

18

```

\NormalTok{\}}

\NormalTok{model SystemLog \{ }
\NormalTok{ { // ... }
\NormalTok{ @@index([createdAt (sort: Desc)]) // dùng cho ORDER BY DESC}
\NormalTok{ @@index([resource, createdAt])}
\NormalTok{ @@index([nguoi\thuc\hien\_id])}
\NormalTok{\}}

\NormalTok{model DanhHieuHangNam \{ }
\NormalTok{ { // ... }
\NormalTok{ @@unique([quan\_nhan\_id, nam]) // composite UNIQUE = composite index}
\NormalTok{\}}

```

SQL DDL tương đương (sinh trong migration.sql)

```

\KeywordTok{CREATE} \KeywordTok{INDEX} \OtherTok{"BangDeXuat\_status\_loai\_de\_xuat\_idx"}
\KeywordTok{ON} \OtherTok{"BangDeXuat"} \NormalTok{(status, loai\_de\_xuat);}

\KeywordTok{CREATE} \KeywordTok{INDEX} \OtherTok{"SystemLog\_createdAt\_desc\_idx"}
\KeywordTok{ON} \OtherTok{"SystemLog"} \NormalTok{(createdAt )\KeywordTok{DESC} \NormalTok{();}

\KeywordTok{CREATE} \KeywordTok{UNIQUE} \KeywordTok{INDEX} \OtherTok{"DanhHieuHangNam\_quan\_nhan\_id\_nam\_key"}
\KeywordTok{ON} \OtherTok{"DanhHieuHangNam"} \NormalTok{(quan\_nhan\_id, nam);}

```

Phát hiện query thiếu index bằng EXPLAIN:

```

\KeywordTok{EXPLAIN} \KeywordTok{ANALYZE}
\KeywordTok{SELECT} \OperatorTok{*} \KeywordTok{FROM} \NormalTok{SystemLog}
\KeywordTok{WHERE} \KeywordTok{resource} \OperatorTok{=} \StringTok{\textquotesingle{ }personnel\textquotesingle{ }}
\KeywordTok{AND} \NormalTok{createdAt} \OperatorTok{\textgreater{}} \NormalTok{NOW() } \OperatorTok{-} \DataTok{ }
\KeywordTok{ORDER} \KeywordTok{BY} \NormalTok{createdAt} \KeywordTok{DESC} \KeywordTok{LIMIT} \DecValTok{30} \NormalTok{ }

```

Output mong đợi:

Index Scan using SystemLog_resource_createdAt_idx ... (đúng)

Nếu thấy Seq Scan trên bảng > 10k row, là dấu hiệu cần thêm index.

Q17. “Connection pool em cấu hình ra sao? Tránh hết pool thế nào?”

Trả lời: Pool mặc định 10 connection, đủ ~50 user đồng thời.

Cấu hình (.env)

```

\VariableTok{DATABASE\_URL} \OperatorTok{=} \StringTok{"postgres://user:pass@localhost:5432/qlkt?connection\_limi"}

```

Pattern release connection sớm:

```

\CommentTok{// SAI: pg\_dump (\textasciitilde{ }30s) chạy trong $transaction {-} \textgreater{ } giữ connection lock
\ControlFlowTok{await} \NormalTok{ prisma} \OperatorTok{.} \FunctionTok{$transaction} \NormalTok{( )} \KeywordTok{async}
\KeywordTok{const} \NormalTok{ data} \OperatorTok{=} \ControlFlowTok{await} \NormalTok{ tx} \OperatorTok{.} \AttribTok{ }
\ControlFlowTok{await} \FunctionTok{spawnPgDump} \NormalTok{( )} \StringTok{\textquotesingle{ }backup.sql\textquotesingle{ }}
\ControlFlowTok{await} \NormalTok{ tx} \OperatorTok{.} \AttribTok{systemLog} \OperatorTok{.} \FunctionTok{create}
\NormalTok{\} } \OperatorTok{;}

```

```

\CommentTok{// ĐÚNG: I/O nặng tách ngoài transaction}
\KeywordTok{const} \NormalTok{ data} \OperatorTok{=} \ControlFlowTok{await} \NormalTok{ prisma} \OperatorTok{.} \AttribTok{ }
\ControlFlowTok{await} \FunctionTok{spawnPgDump} \NormalTok{( )} \StringTok{\textquotesingle{ }backup.sql\textquotesingle{ }}
\ControlFlowTok{await} \NormalTok{ prisma} \OperatorTok{.} \AttribTok{systemLog} \OperatorTok{.} \FunctionTok{create}

```

Monitor pool exhaustion:

```

\CommentTok{{-}{-} Check Postgres bao nhiêu connection từ backend đang active}
\KeywordTok{SELECT} \NormalTok{ pid, username, application\_name, state, query\_start, state\_change}
\KeywordTok{FROM} \NormalTok{ pg\_stat\_activity}
\KeywordTok{WHERE} \NormalTok{ application\_name} \KeywordTok{LIKE} \StringTok{\textquotesingle{ }prisma\%\textquotesingle{ }}
\KeywordTok{ORDER} \KeywordTok{BY} \NormalTok{ state\_change;}

```

Nếu thấy nhiều state idle in transaction lâu — đang có transaction chạy quá dài → cần optimize.

Q18. “Tại sao em không dùng MongoDB hay NoSQL?”

Trả lời: PostgreSQL hỗ trợ JSONB, vẫn được linh hoạt schema mà không mất ACID + relational.

Schema Prisma (cột JSONB)

```
\NormalTok{model BangDeXuat \{\}  
\NormalTok{  id          String @id}  
\NormalTok{  loai\_de\_xuat  String}  
\NormalTok{  data\_danh\_hieus Json?    // JSONB column}  
\NormalTok{  data\_thanh\_tich Json?}  
\NormalTok{  data\_nien\_han   Json?}  
\NormalTok{  data\_cong\_hien   Json?}  
\NormalTok{\}
```

Truy vấn JSONB như NoSQL:

```
\CommentTok{// Prisma JSON path filter}  
\NormalTok{prisma}\OperatorTok{.}\AttributeTok{bangDeXuat}\OperatorTok{.}\FunctionTok{findMany}\NormalTok{\{\}  
\NormalTok{  where}\OperatorTok{:}\NormalTok{\{\}  
\NormalTok{    data\_danh\_hieus}\OperatorTok{:}\NormalTok{\{\}  
\NormalTok{      path}\OperatorTok{:}\NormalTok{\{\StringTok{\textquotesingle}ranking\textquotesingle}\NormalTok{\}  
\NormalTok{      equals}\OperatorTok{:}\StringTok{\textquotesingle}gold\textquotesingle}\OperatorTok{,}  
\NormalTok{    }\}\OperatorTok{,}  
\NormalTok{  }\}\OperatorTok{,}  
\NormalTok{\}\}\OperatorTok{;}
```

SQL tương đương (PostgreSQL JSONB operator):

```
\KeywordTok{SELECT} \OperatorTok{*} \KeywordTok{FROM} \OtherTok{"BangDeXuat"}  
\KeywordTok{WHERE} \NormalTok{data\_danh\_hieus}\OperatorTok{-}\StringTok{\textquoteleft}\NormalTok{\}  
  
\CommentTok{\{-}\{-} Hoặc with index trên JSONB key:  
\KeywordTok{CREATE} \KeywordTok{INDEX} \NormalTok{idx\_dh\_ranking} \KeywordTok{ON} \OtherTok{"BangDeXuat"} \KeywordTok{USING}
```

MongoDB tương đương (chỉ minh họa — em không dùng):

```
\NormalTok{db}\OperatorTok{.}\AttributeTok{proposals}\OperatorTok{.}\FunctionTok{find}\NormalTok{\{\{\StringTok{\textquoteleft}
```

PostgreSQL có cả 2 ưu điểm: ACID + relational + JSONB linh hoạt như MongoDB.

Q19. “Em làm sao đảm bảo log kiểm toán không bị xóa hay sửa?”

Trả lời: 3 lớp bảo vệ.

Lớp 1 — Code chỉ INSERT, không có route UPDATE:

```
\CommentTok{// repositories/systemLog.repository.ts — chỉ có create}  
\ImportTok{export} \KeywordTok{const} \NormalTok{systemLogRepository} \OperatorTok{=}\NormalTok{\{\}  
\NormalTok{  create}\OperatorTok{:}\NormalTok{\(data)\OperatorTok{,}\NormalTok{tx}\OperatorTok{=}\NormalTok{\}  
\NormalTok{  findMany}\OperatorTok{:}\NormalTok{\(where)\OperatorTok{,}\NormalTok{tx}\OperatorTok{=}\NormalTok{\}  
\NormalTok{  cleanup}\OperatorTok{:}\NormalTok{\(cutoff)\OperatorTok{,}\NormalTok{tx}\OperatorTok{=}\NormalTok{\}  
\NormalTok{  where}\OperatorTok{:}\NormalTok{\{ createdAt}\OperatorTok{:}\NormalTok{\{ lt}\OperatorTok{:}\NormalTok{\}  
\NormalTok{\}\}\OperatorTok{,}  
\CommentTok{// KHÔNG có update / hardDelete}  
\NormalTok{\}\}\OperatorTok{;}
```

Lớp 2 — Database REVOKE quyền UPDATE/DELETE cho user backend:

```
\CommentTok{\{-}\{-} Tạo role riêng cho backend}  
\KeywordTok{CREATE} \FunctionTok{USER} \NormalTok{qlkt\_app} \KeywordTok{WITH} \KeywordTok{PASSWORD} \StringTok{\}  
\KeywordTok{GRANT} \KeywordTok{SELECT} \NormalTok{,} \KeywordTok{INSERT} \KeywordTok{ON} \NormalTok{SystemLog} \KeywordTok{TO} \NormalTok{\}  
\CommentTok{\{-}\{-} KHÔNG grant UPDATE/DELETE}  
\KeywordTok{GRANT} \KeywordTok{DELETE} \KeywordTok{ON} \NormalTok{SystemLog} \KeywordTok{TO} \NormalTok{qlkt\_adm}
```

Lớp 3 — Partition theo tháng + READ-ONLY partition cũ:

```

\CommentTok{{-}{-} Convert SystemLog thành partitioned table theo createdAt}
\KeywordTok{CREATE} \KeywordTok{TABLE}\NormalTok{ SystemLog\_2026\_05 }\KeywordTok{PARTITION} \KeywordTok{OF}\NormalTok{
  \ControlFlowTok{FOR} \KeywordTok{VALUES} \KeywordTok{FROM}\NormalTok{ ( )\StringTok{\textquotesingle{}2026{-}05}
\CommentTok{{-}{-} Sau khi tháng kết thúc, set partition cũ READ{-}ONLY}
\KeywordTok{ALTER} \KeywordTok{TABLE}\NormalTok{ SystemLog\_2026\_04 }\KeywordTok{SET}\NormalTok{ (autovacuum\_en
\KeywordTok{REVOKE} \KeywordTok{INSERT}\NormalTok{ {, } }\KeywordTok{UPDATE}\NormalTok{ {, } }\KeywordTok{DELETE} \KeywordTok{

```

Hiện em mới làm Lớp 1; Lớp 2 và 3 là hướng phát triển ghi tại Chương 6.

Q20. “Race condition khi 2 user cùng tạo quân nhân với cùng CCCD?”

Trả lời: Postgres UNIQUE constraint chặn ở lớp DB, không cần lock.

Schema Prisma

```

\NormalTok{model QuanNhan \{\}
\NormalTok{ id String @id @default(cuid())}
\NormalTok{ cccd String @unique // \textless{}{-}{-} UNIQUE constraint}
\NormalTok{ // ...}
\NormalTok{\}}

```

SQL DDL

```

\KeywordTok{CREATE} \KeywordTok{UNIQUE} \KeywordTok{INDEX} \OtherTok{"QuanNhan\_cccd\_key"} \KeywordTok{ON} \OtherTok{

```

Behavior: 2 INSERT đồng thời với cùng cccd:

```

\CommentTok{{-}{-} T1 (trước): INSERT INTO QuanNhan (cccd, ...) VALUES (\textquotesingle{}123\textquotesingle{}),
\CommentTok{{-}{-} T2 (cùng lúc): INSERT INTO QuanNhan (cccd, ...) VALUES (\textquotesingle{}123\textquotesingle{}),
\CommentTok{{-}{-} ERROR: duplicate key value violates unique constraint "QuanNhan\_cccd\_key"}
\CommentTok{{-}{-} DETAIL: Key (cccd)=(123) already exists.}
\CommentTok{{-}{-} SQLSTATE: 23505}

```

Backend bắt lỗi P2002 của Prisma:

```

\ControlFlowTok{try}\NormalTok{ \{\}
  \ControlFlowTok{await}\NormalTok{ prisma}\OperatorTok{.}\AttributeTok{quanNhan}\OperatorTok{.}\FunctionTok{create}
\NormalTok{\} }\ControlFlowTok{catch}\NormalTok{ (e) \{\}
  \ControlFlowTok{if}\NormalTok{ (e)\KeywordTok{instanceof}\NormalTok{ Prisma}\OperatorTok{.}\AttributeTok{Prisma}
    \ControlFlowTok{throw} \KeywordTok{new} \FunctionTok{ConflictError}\NormalTok{ (}\StringTok{\textquotesingle{}
\NormalTok{\} \{\}
  \ControlFlowTok{throw}\NormalTok{ e}\OperatorTok{;}
\NormalTok{\} }

```

Test verify (tests/concurrent/createPersonnel.test.ts):

```

\FunctionTok{it}\NormalTok{ (}\StringTok{\textquotesingle{}chi 1 trong 2 INSERT đồng thời thành công\textquotesingle{}
  \KeywordTok{const}\NormalTok{ data }\OperatorTok{=}\NormalTok{ \{\} cccd}\OperatorTok{:}\StringTok{\textquotesingle{}
  \KeywordTok{const}\NormalTok{ results }\OperatorTok{=}\ControlFlowTok{await} \BuiltInTok{Promise}\OperatorTok{.}
    \FunctionTok{createPersonnel}\NormalTok{ (data)}\OperatorTok{,}
    \FunctionTok{createPersonnel}\NormalTok{ (data)}\OperatorTok{,}
\NormalTok{ })}\OperatorTok{;}
  \KeywordTok{const}\NormalTok{ fulfilled }\OperatorTok{=}\NormalTok{ results}\OperatorTok{.}\FunctionTok{filter}
  \KeywordTok{const}\NormalTok{ rejected }\OperatorTok{=}\NormalTok{ results}\OperatorTok{.}\FunctionTok{filter}
  \FunctionTok{expect}\NormalTok{ (fulfilled)\OperatorTok{.}\AttributeTok{length}\NormalTok{ () }\OperatorTok{.}\FunctionTok{
  \FunctionTok{expect}\NormalTok{ (rejected)\OperatorTok{.}\AttributeTok{length}\NormalTok{ () }\OperatorTok{.}\FunctionTok{
  \FunctionTok{expect}\NormalTok{ (rejected)\DecValTok{0}\NormalTok{ () }\OperatorTok{.}\AttributeTok{reason}\NormalTok{
\NormalTok{\} }\OperatorTok{;}

```

13. Cheat sheet — phải nhớ thuộc lòng

Khái niệm	Cú pháp Prisma	SQL
Lấy 1 bản ghi	<code>findUnique({where:{id}})</code>	<code>SELECT ... WHERE id = \$1</code>
Lọc nhiều điều kiện	<code>findMany({where:{AND:[...]}\$1}</code>	<code>SELECT ... WHERE a AND b</code>

Khái niệm	Cú pháp Prisma	SQL
OR	where:{OR:[{a},{b}]}	WHERE a OR b
LIKE	contains: 's', mode:'insensitive'	ILIKE '%s%'
JOIN	include:{Rel:true}	LEFT JOIN ... ON ...
Phân trang	skip, take	OFFSET, LIMIT
Đếm	count({where})	SELECT COUNT(*)
Group by	groupBy({by:[col], _count:{}})	GROUP BY
Upsert	upsert({where, create, update})	INSERT ... ON CONFLICT DO UPDATE
Transaction	\$transaction(async tx => ...)	BEGIN; ... COMMIT;
Tăng/giảm	data:{so_luong:{increment:\$1}}	SET so_luong = so_luong + 1
In list	where:{id:{in:[...]}}	WHERE id IN (...)
Date range	createdAt:{gte:from, lt:to}	WHERE createdAt >= \$1 AND createdAt < \$2
Sắp xếp nhiều cột	orderBy:[{a:'desc'}, {b:'asc'}]	ORDER BY a DESC, b ASC